

CodebookAI: LLM-powered deductive coding for qualitative research

Torsten Maier ¹

¹ Kettering University, United States

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 14 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

CodebookAI is a desktop application for qualitative researchers that automates deductive coding—the systematic assignment of predefined category labels to text segments—using large language models (LLMs). Researchers in social sciences, education, health sciences, and related disciplines routinely analyze interview transcripts, open-ended survey responses, and similar text by applying codes from a structured codebook (Saldaña, 2021). This process is labor-intensive when datasets contain hundreds or thousands of segments.

CodebookAI provides a Tkinter-based graphical interface through which researchers upload a codebook (a flat list of category names) and text segments from CSV, TSV, Excel, or Parquet files. The application constructs structured prompts, enforces codebook fidelity through JSON-schema-constrained outputs validated by Pydantic, and returns coded results as spreadsheets ready for analysis in tools such as R, SPSS, or Excel.

The application supports two processing modes—**batch** and **live**—and four primary workflows: single-label classification, multi-label classification, keyword extraction, and inter-rater reliability analysis. A **correlogram** tool visualizes pairwise co-occurrence between two sets of codes as a heatmap, and a **data preparation** module enables stratified random sampling to create representative coding subsets for pilot studies.

Statement of need

Deductive qualitative coding is a methodological cornerstone of qualitative content analysis (Mayring, 2022) and systematic qualitative inquiry more broadly. The process requires trained coders to evaluate each text segment against a codebook, and often involves two or more coders to establish inter-rater reliability. For datasets of several hundred to several thousand items, this represents a substantial investment of researcher time.

CodebookAI addresses this problem by giving researchers a graphical tool that applies LLM-based classification to user-defined codebooks without requiring programming knowledge. It is intended for researchers who need structured, reproducible assistance with coding large collections of text segments while preserving a codebook-driven workflow. The software supports both exploratory pilot work on small samples and large-scale studies involving tens of thousands of segments processed through batch inference.

State of the field

Researchers currently have several imperfect options for deductive coding assistance. Manual coding in qualitative data analysis platforms such as NVivo, ATLAS.ti, and MAXQDA supports established research workflows, but these tools are centered on human coding rather than LLM-based automated classification. General-purpose LLM interfaces such as ChatGPT can

39 classify text, but they do not provide integrated codebook management, structured export,
40 batch submission workflows, or built-in reliability analysis suitable for systematic research
41 projects.

42 Other computational alternatives also have important limitations. Keyword-based approaches
43 are brittle for semantically complex text, especially when categories depend on context rather
44 than explicit vocabulary. Supervised machine-learning approaches can be effective, but they
45 require labeled training data, model development expertise, and a different workflow from
46 the zero-shot or instruction-based classification that LLMs make possible. CodebookAI's
47 contribution is to bridge this gap by combining a researcher-facing desktop interface with
48 structured-output constraints, batch and live processing, spreadsheet-oriented import/export,
49 and post-classification reliability tools in a single application.

50 Software design

51 CodebookAI is implemented as a Tkinter-based desktop application designed for researchers
52 who may not have programming experience but still need transparent and repeatable coding
53 workflows. A central design choice is the separation between the user interface, the prompt-
54 and-schema generation logic, and the API interaction layer. This structure supports both
55 interactive use and testability while allowing the software to expose a simple GUI.

56 A second major design choice is the use of runtime-generated structured outputs. Rather than
57 allowing the model to return arbitrary free text, CodebookAI builds a Pydantic model from the
58 user-supplied codebook, derives a JSON schema from that model, and uses the schema to
59 constrain outputs from the API (OpenAI, 2023). This design reduces invalid labels and keeps
60 the model aligned with the researcher's predefined category set.

61 The software also supports both **live** and **batch** execution modes because these modes serve
62 different research needs. Live mode is useful for smaller jobs and iterative exploration, while
63 batch mode is better suited for large studies where lower cost and higher throughput are more
64 important than immediate responses. Additional modules for inter-rater reliability, correlograms,
65 and random sampling were included so that the application supports more of the end-to-end
66 research workflow rather than limiting itself to a single classification step.

67 Functionality

68 Users launch CodebookAI without installing dependencies beyond the single-file Windows
69 executable, or alternatively from source in a standard Python environment. After entering an
70 OpenAI API key in **File** → **Settings**, the workflow proceeds through data import, codebook
71 selection, model execution, and export of results.

72 For classification, a file import wizard accepts CSV, TSV, Excel, and Parquet files. The user
73 selects the column containing text segments and imports a codebook as a flat list of labels.
74 CodebookAI builds a Pydantic model at runtime that treats the label list as a strict enumeration,
75 derives a JSON schema from it, and passes the schema to the API as a structured-output
76 constraint. In batch mode, requests are bundled into a JSONL file and submitted through
77 OpenAI's Batch API; in live mode, segments are classified sequentially with progress reporting.
78 Results are exported as CSV.

79 The same general workflow supports **multi-label classification** and **keyword extraction**, while the
80 **inter-rater reliability** module compares two coded datasets and computes matched rows, percent
81 agreement, and Cohen's kappa (Cohen, 1960), exporting results to Excel. The **correlogram**
82 tool visualizes cross-dataset code relationships as a heatmap, and the **data preparation** module
83 supports random sampling to create pilot subsets before running full-scale coding jobs.

Research impact statement

CodebookAI has already been used to support published research. In particular, the software was used in the study *Examining the Reliability of ChatGPT: A Quantitative Comparison of Human and LLM Emotion Classification*, published through the American Society for Engineering Education (ASEE) proceedings, which investigated the reliability of large language models in an emotion-classification task (Maier et al., 2025). This provides direct evidence that the software has moved beyond a prototype and has already supported an actual research workflow and scholarly output.

The software is designed to be useful in additional research settings where investigators need to classify large collections of open-ended text with a predefined codebook and then evaluate agreement or downstream patterns. Its support for spreadsheet-based inputs and outputs, structured classification constraints, and integrated reliability analysis lowers the practical barrier for researchers who want to incorporate LLM-assisted coding into empirical studies without building custom software pipelines.

AI usage disclosure

GitHub Copilot was used extensively during software development, generating the majority of front-end code and portions of the back-end implementation. The core classification logic, structured-output schema generation, and overall architecture were authored by the human developer. All Copilot-generated code was reviewed, validated, debugged, and integrated by the human author, who takes full responsibility for the software's correctness, licensing, and performance.

GitHub Copilot was also used to generate automated tests, CI workflows, and the CONTRIBUTING.md file. All AI-generated content was reviewed and substantially revised for accuracy by the author.

Acknowledgments

The author thanks the students and research collaborators at Kettering University whose qualitative research needs motivated the development of this tool.

References

- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46. <https://doi.org/10.1177/001316446002000104>
- Maier, T., Mendez, S., & Kneebone, S. (2025). Examining the reliability of ChatGPT: A quantitative comparison of human and LLM emotion classification. *2026 ASEE North Central Section Conference*. <https://doi.org/10.18260/1-2--58351>
- Mayring, P. (2022). Qualitative content analysis: A step-by-step guide. *Sage*, 1(2). <https://doi.org/10.4135/9781036231798>
- OpenAI. (2023). *GPT-4 technical report*. OpenAI. <https://arxiv.org/abs/2303.08774>
- Saldaña, J. (2021). *The coding manual for qualitative researchers* (4th ed.). SAGE Publications. <https://doi.org/10.4135/9781036235611>